Szoftvertesztelés Projekt Dokumentáció

Unit Testing with Pytest

Készítette: Balázs Ákos
Csoport: Számítástechnika IV.B

Sapientia Erdélyi Magyar Tudományegyetem
2026

Tartalomjegyzék

1. Bevezetés **2**

2. Választott tesztkörnyezet **2**
2.1. Motivációk 2
2.2. Telepítés és beállítás 2

3. A tesztelt kódbázis felépítése **2**

4. Megvalósított Unit Tesztek **2**
4.1. RelationsManager tesztelése 2
4.2. Tesztesetek leírása 3

5. Kihívások a tesztelés során: Mockolás **3**
5.1. Az időfüggőség kezelése (Datetime Mocking) 3
5.2. Függőségek és külső szolgáltatások szimulálása 4
5.3. EmployeeManager Tesztesetek 4

6. Folyamatos Integráció (CI/CD) **5**

7. Összegzés **5**

1. Bevezetés

A projekt célja a szoftvertesztelés alapfogalmainak elsajátítása egy egyszerűsített alkalmazott-kezelő (Employee Management) rendszeren keresztül. Elsődleges fókuszom a unit tesztelés, melynek során izolált kódegységeket (függvényeket, osztályokat) vizsgálok meg.

2. Választott tesztkörnyezet

A projekt megvalósításához a **Python** nyelvet és a **pytest** keretrendszert választottam a feladat ajánlása alapján.

2.1. Motivációk

- A pytest az egyik legelterjedtebb nyílt forráskódú tesztelési keretrendszer Pythonhoz.
- Egyszerű és kompakt tesztszvitek létrehozását teszi lehetővé.
- Rendkívül bővíthető pluginekkel (pl. pytest-cov, pytest-django).
- Aktív közösségi támogatással rendelkezik.

2.2. Telepítés és beállítás

A fejlesztői környezet kialakítása a következő lépésekben történt:

1. Virtuális környezet létrehozása: `python -m venv pytest-env`.
2. Környezet aktiválása Windows alatt: `pytest-env\Scripts\activate`.
3. A keretrendszer telepítése: `pip install pytest`.

3. A tesztelt kódbázis felépítése

A kapott szoftver három fő komponensből áll:

- **Employee**: Adatmodell az alkalmazottak tárolására (id, név, fizetés, dátumok).
- **RelationsManager**: Kezeli a szervezeti hierarchiát és a csapatokat.
- **EmployeeManager**: Felelős a fizetések kiszámításáért és az értesítések küldéséért.

4. Megvalósított Unit Tesztek

4.1. RelationsManager tesztelése

Ebben a szakaszban a `test_relations_manager.py` fájlban definiált teszteket mutatom be. A tesztek során ellenőrzöm a csapatvezetők meglétét, a csapattagok listáját és a hibakezelést.

```python
import pytest
import datetime
from relations_manager import RelationsManager

@pytest.fixture
def rm():
    return RelationsManager()

def get_employee_by_name(rm, first_name, last_name):
    for e in rm.get_all_employees():
        if e.first_name == first_name and e.last_name == last_name:
            return e
    return None

def test_john_doe_is_leader_and_birthdate(rm):
    john = get_employee_by_name(rm, "John", "Doe")
    assert john is not None
    assert rm.is_leader(john) is True
    assert john.birth_date == datetime.date(1970, 1, 31)
```

Listing 1. A RelationsManager tesztjeinek kezdete

4.2. Tesztesetek leírása

Teszt neve	Bemenet	Elvárt kimenet
John Doe adatai	John Doe lekérdezése	Leader: True, Szül: 1970.01.31
Csapattagok ellenőrzése	John Doe csapata	Myrta Torkelson, Jettie Lynch
Nem létező dolgozó	"Jude Overcash"	None

1. táblázat. RelationsManager tesztesetek összefoglalása

5. Kihívások a tesztelés során: Mockolás

Az `EmployeeManager` osztály tesztelése során két fő problémába ütköztem, amelyek megoldásához a Python beépített `unittest.mock` könyvtárát használtam.

5.1. Az időfüggőség kezelése (Datetime Mocking)

A fizetésszámítás logikája a mindenkori mai évet (`datetime.date.today().year`) használja a munkaviszony hosszának megállapításához. Mivel a feladat specifikációja fix elvárt értékeket (pl. 3000$) határozott meg egy 1998-ban felvett dolgozó esetén (20 év munkaviszonyt feltételezve), a tesztkörnyezetben „be kellett fagyasztanom” az időt 2018-ra.

Ezt egy saját `MockDate` osztály létrehozásával és a `@patch` dekorátor alkalmazásával értem el:

```python
class MockDate(datetime.date):
    @classmethod
    def today(cls):
        return cls(2018, 1, 1)

@patch('employee_manager.datetime.date', MockDate)
```

```python
def test_regular_employee_salary(em):
    emp = Employee(
        id=99, first_name="Regular", last_name="Joe", base_salary=1000,
        birth_date=datetime.date(1980, 1, 1),
        hire_date=datetime.date(1998, 10, 10)
    )
    salary = em.calculate_salary(emp)
    assert salary == 3000
```

Listing 2. Az idő fagyasztása a tesztekhez

5.2. Függőségek és külső szolgáltatások szimulálása

A feladat megkövetelte egy e-mail küldő szolgáltatás tesztelését, amely a kódbázisban egy egyszerű `print` utasításként volt implementálva. Valós e-mailek küldése helyett a `@patch('builtins.print')` segítségével „lehallgattam” a standard kimenetet, és ellenőriztem, hogy a megfelelő paraméterekkel hívták-e meg.

Továbbá az adatbázisban nem szerepelt olyan vezető, akinek pontosan 3 beosztottja lett volna, így a `RelationsManager` metódusait felülírtam a teszt idejére (`MagicMock`):

```python
@patch('builtins.print')
@patch('employee_manager.datetime.date', MockDate)
def test_salary_notification(mock_print, em, rm):
    leader = Employee(id=100, first_name="Boss", last_name="Lady",
                      base_salary=2000, birth_date=datetime.date(1980, 1, 1),
                      hire_date=datetime.date(2008, 10, 10))

    # Kapcsolatok szimul l sa (3 f s csapat)
    rm.is_leader = MagicMock(return_value=True)
    rm.get_team_members = MagicMock(return_value=[101, 102, 103])

    em.calculate_salary_and_send_email(leader)

    # Assertions
    mock_print.assert_called_once()
    expected_msg = "Boss Lady your salary: 3600 has been transferred to you."
    mock_print.assert_called_with(expected_msg)
```

Listing 3. Metódusok és a print függvény mockolása

5.3. EmployeeManager Tesztesetek

Teszt neve	Bemenet / Feltétel	Elvárt kimenet
Sima dolgozó fizetése	Belépés: 1998, Alapfizetés: 1000$	3000$
Vezető fizetése (3 tag)	Belépés: 2008, Alapfizetés: 2000$, 3 csapattag	3600$
Értesítés küldése	Fizetéskalkuláció lefutása a vezetőn	Helyes üzenet printelése

2. táblázat. EmployeeManager tesztesetek összefoglalása

6. Folyamatos Integráció (CI/CD)

A tesztelés automatizálása érdekében a projekt repository-jában beállítottam a GitHub Actions-t. Létrehoztam egy `.github/workflows/python-tests.yml` fájlt, amely minden új kód feltöltésekor (push) vagy Pull Request indításakor automatikusan felépít egy Ubuntu alapú virtuális környezetet, telepíti a `pytest` keretrendszert, és lefuttatja az összes tesztet. Ez biztosítja a kód folyamatos minőségellenőrzését és a regressziós hibák azonnali detektálását.

7. Összegzés

A projekt során sikeresen implementáltam a követelményekben meghatározott unit teszteket. A `pytest` keretrendszer, valamint a fixture-ök és a mocking technikák alkalmazásával egy robusztus, a külső függőségektől (pl. aktuális dátum, e-mail szerver) elszigetelt tesztszvitet hoztam létre. A megfelelő projektstruktúra és a verziókezelés (Git, .gitignore) használata nagyban hozzájárult a kód karbantarthatóságához.